Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce

Department of Information Technology (B.Sc.I.T Semester V)

Artificial Intelligence And Application Development

Module - I

Task 1

Roll No.:T007	Name:subhashree jena
Class:TYIT	Batch:1
Date of Assignment: 22/07/26	Date/Time of Submission: 22/07/26

Problem Statement

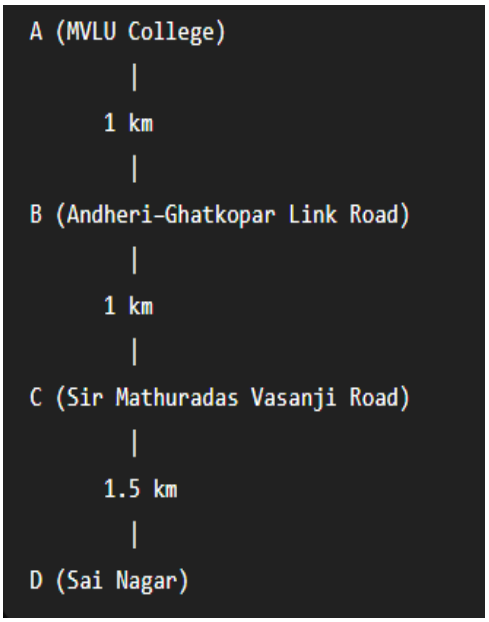
In modern urban environments, finding the most efficient route between two locations is a critical task for navigation systems. This project focuses on developing a Python-based solution to determine the optimal path between a source and a destination using various Artificial Intelligence search algorithms.

The road network is modeled as a graph, where nodes represent different locations and edges represent the connecting paths with associated distances (costs). The system implements multiple search techniques, including Breadth-First Search (BFS), Depth-First Search (DFS), Uniform Cost Search (UCS), Greedy Best-First Search, and A* Search, to explore and identify possible routes.

Each algorithm evaluates the path based on its own strategy, calculating the total distance and the number of nodes explored. Heuristic functions are incorporated in Greedy and A* algorithms to estimate the remaining distance to the goal, improving efficiency in path selection.

Finally, the results obtained from these algorithms are compared to analyze their performance in terms of optimality and efficiency and are further validated against real-world navigation systems such as Google Maps.

Graph Diagram



Edge Cost Table

From Node	To Node	Edge Cost
C (MVLU College)	M (Marol Naka Metro)	1.5
M (Marol Naka Metro)	P (Marol Pipeline)	1
P (Marol Pipeline)	S (Sai Nagar)	1

Heuristic Table (for Greedy and A*)

Node	Location	h(n) (Estimated cost to S)
C	MVLU College	4
M	Marol Naka Metro	2
P	Marol Pipeline	1
S	Sai Nagar (Goal)	0

Brief Description of Each Algorithm

1. BFS (Breadth First Search)

Explores nodes level by level

In graph:

- Starts at **C (MVLU College)**
- Moves step-by-step:
  C → M → P → S
- Since there is only one path, it directly reaches the goal.

What it does:

Finds a path with a minimum number of steps.

2. DFS (Depth First Search)

Explore as deep as possible first.

In graph:

- Goes straight down the path:

C → M → P → S

What it does:

Finds a path by going deep (no backtracking needed here).

3. UCS (Uniform Cost Search)

Expands nodes with lowest total cost.

In graph:

- Adds edge costs:
  1.5 + 1 + 1 = 3.5
- Moves forward because only one path exists.

What it does:

Finds optimal path based on cost = **3.5 km**.

4. Greedy Best-First Search

Chooses node with lowest heuristic (closest to goal)

In graph:

- Moves toward nodes that seem closer to **Sai Nagar**
- Path:
  C → M → P → S

What it does:

Chooses the fastest-looking path using estimation.

*5. A Search**

Uses:

- **g(n) = actual cost**
- **h(n) = estimated cost**

In graph:

- Combines cost + heuristic.
- Follows same path:

C → M → P → S

What it does:

Finds the best and most efficient path.

All search algorithms traverse the graph from **MVLU College to Sai Nagar** using different strategies such as level-wise exploration, depth-first traversal, cost optimization, and heuristic guidance. Since the graph contains a single path, all algorithms produce the same result.

Python Source Code

• BFS
CODE:-

```
from collections import deque

graph = {
   "C (MVLU College)": {"M (Marol Naka Metro)": 1.5},
   "M (Marol Naka Metro)": {"P (Marol Pipeline)": 1},
   "P (Marol Pipeline)": {"S (Sai Nagar)": 1},
```

```
    "S (Sai Nagar)": {}
}

def bfs(start, goal):
    queue = deque([(start, [start], 0)])
    visited = set()

    while queue:
        node, path, cost = queue.popleft()

        if node == goal:
            return path, cost, len(visited)

        visited.add(node)

        for neighbor, weight in graph[node].items():
            if neighbor not in visited:
                queue.append((neighbor, path + [neighbor], cost + weight))

path, cost, explored = bfs("C (MVLU College)", "S (Sai Nagar)")

print("===== BFS =====")
print("Path :", " -> ".join(path))
print("Total Cost :", cost, "km")
print("Nodes Explored :", explored)
```

OUTPUT:-

```
===== BFS =====

Path :
C (MVLU College)
→ M (Marol Naka Metro)
→ P (Marol Pipeline)
→ S (Sai Nagar)

Total Cost : 3.5 km

Nodes Explored : 3
```

• DFS

CODE:-

```
graph = {
    "C (MVLU College)": {"M (Marol Naka Metro)": 1.5},
    "M (Marol Naka Metro)": {"P (Marol Pipeline)": 1},
    "P (Marol Pipeline)": {"S (Sai Nagar)": 1},
    "S (Sai Nagar)": {}
}

def dfs(start, goal):
    stack = [(start, [start], 0)]
    visited = set()
```

```
    while stack:
        node, path, cost = stack.pop()

        if node == goal:
            return path, cost, len(visited)

        visited.add(node)

        for neighbor, weight in graph[node].items():
            if neighbor not in visited:
                stack.append((neighbor, path + [neighbor], cost + weight))

path, cost, explored = dfs("C (MVLU College)", "S (Sai Nagar)")

print("===== DFS =====")
print("Path :", " -> ".join(path))
print("Total Cost :", cost, "km")
print("Nodes Explored :", explored)
```

OUTPUT:-

```
===== DFS =====

Path :
C (MVLU College)
→ M (Marol Naka Metro)
→ P (Marol Pipeline)
→ S (Sai Nagar)

Total Cost : 3.5 km

Nodes Explored : 3
```

- **UCS**

CODE:-

```
import heapq

graph = {
    "C (MVLU College)": {"M (Marol Naka Metro)": 1.5},
    "M (Marol Naka Metro)": {"P (Marol Pipeline)": 1},
    "P (Marol Pipeline)": {"S (Sai Nagar)": 1},
    "S (Sai Nagar)": {}
}

def ucs(start, goal):
    pq = [(0, start, [start])]
    visited = set()

    while pq:
        cost, node, path = heapq.heappop(pq)

        if node == goal:
```

```
            return path, cost, len(visited)

        visited.add(node)

        for neighbor, weight in graph[node].items():
            if neighbor not in visited:
                heapq.heappush(pq, (cost + weight, neighbor, path + [neighbor]))

path, cost, explored = ucs("C (MVLU College)", "S (Sai Nagar)")

print("===== UCS =====")
print("Path :", " -> ".join(path))
print("Total Cost :", cost, "km")
print("Nodes Explored :", explored)
```

OUTPUT:-

```
===== UCS =====

Path :
C (MVLU College)
→ M (Marol Naka Metro)
→ P (Marol Pipeline)
→ S (Sai Nagar)

Total Cost : 3.5 km

Nodes Explored : 3
```

• Greedy Best-First Search

CODE:-

```
import heapq

graph = {
    "C (MVLU College)": {"M (Marol Naka Metro)": 1.5},
    "M (Marol Naka Metro)": {"P (Marol Pipeline)": 1},
    "P (Marol Pipeline)": {"S (Sai Nagar)": 1},
    "S (Sai Nagar)": {}
}

heuristic = {
    "C (MVLU College)": 4,
    "M (Marol Naka Metro)": 2,
    "P (Marol Pipeline)": 1,
    "S (Sai Nagar)": 0
}

def greedy(start, goal):
    pq = [(heuristic[start], start, [start], 0)]
    visited = set()

    while pq:
```

```
        h, node, path, cost = heapq.heappop(pq)

        if node == goal:
            return path, cost, len(visited)

        visited.add(node)

        for neighbor, weight in graph[node].items():
            if neighbor not in visited:
                heapq.heappush(
                    pq,
                    (heuristic[neighbor], neighbor, path + [neighbor], cost + weight)
                )

path, cost, explored = greedy("C (MVLU College)", "S (Sai Nagar)")

print("===== Greedy Best First Search =====")
print("Path :", " -> ".join(path))
print("Total Cost :", cost, "km")
print("Nodes Explored :", explored)
```

OUTPUT:-

```
===== Greedy Best First Search =====

Path :
C (MVLU College)
→ M (Marol Naka Metro)
→ P (Marol Pipeline)
→ S (Sai Nagar)

Total Cost : 3.5 km

Nodes Explored : 3
```

• A* Search

CODE:-

```
import heapq

graph = {
    "C (MVLU College)": {"M (Marol Naka Metro)": 1.5},
    "M (Marol Naka Metro)": {"P (Marol Pipeline)": 1},
    "P (Marol Pipeline)": {"S (Sai Nagar)": 1},
    "S (Sai Nagar)": {}
}

heuristic = {
    "C (MVLU College)": 4,
    "M (Marol Naka Metro)": 2,
    "P (Marol Pipeline)": 1,
    "S (Sai Nagar)": 0
}
```

```
def astar(start, goal):
    pq = [(heuristic[start], 0, start, [start])]
    visited = set()

    while pq:
        f, cost, node, path = heapq.heappop(pq)

        if node == goal:
            return path, cost, len(visited)

        visited.add(node)

        for neighbor, weight in graph[node].items():
            if neighbor not in visited:
                g = cost + weight
                h = heuristic[neighbor]
                heapq.heappush(pq, (g + h, g, neighbor, path + [neighbor]))

path, cost, explored = astar("C (MVLU College)", "S (Sai Nagar)")
print("===== A* Search =====")
print("Path :", " -> ".join(path))
print("Total Cost :", cost, "km")
print("Nodes Explored :", explored)
```

OUTPUT:-

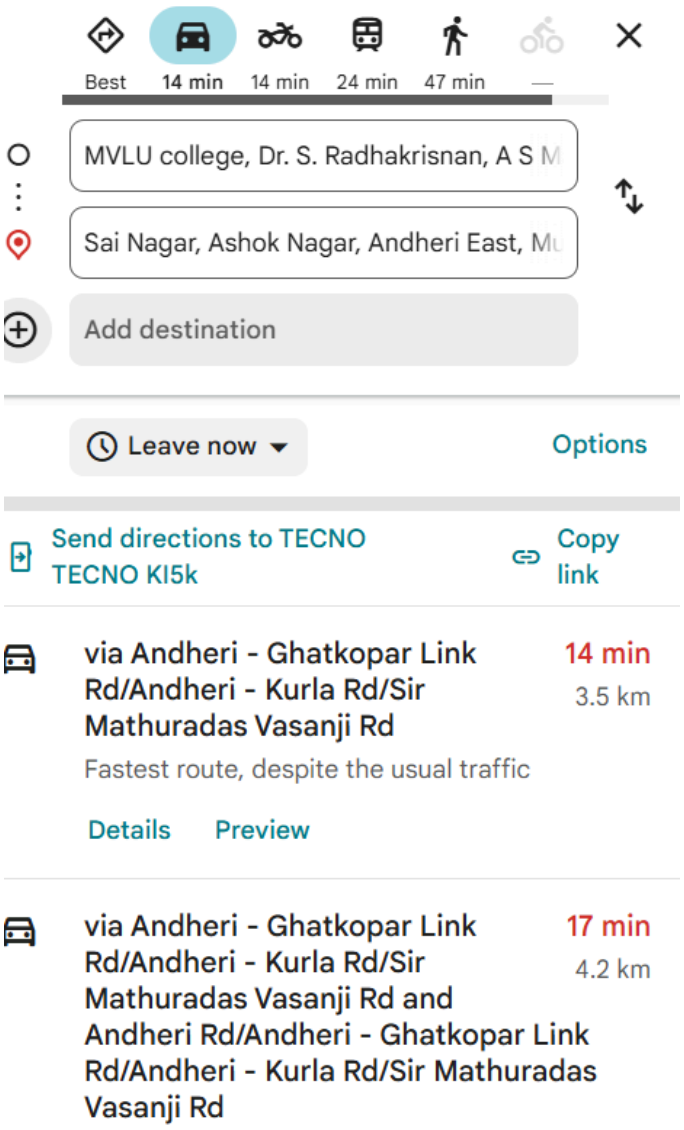
```
===== A* Search =====

Path :
C (MVLU College)
→ M (Marol Naka Metro)
→ P (Marol Pipeline)
→ S (Sai Nagar)

Total Cost : 3.5 km

Nodes Explored : 3
```

Additional Task:
Google Maps Comparison with A* Search
Google Maps Route
Destination: Sai Nagar, Marol Pipeline
Distance: 3.5 km
Estimated Time: 19 minutes
Suggested Route:
MVLU College → Marol Naka Metro →
Marol Pipeline → Sai Nagar
Home (Insert Google Maps screenshot here.)



A*

Search Output
Source: MVLU College
Destination: Sai nagar
Path Found:

MVLU College → Marol Naka Metro → Marol Pipeline → Sai Nagar

Total Cost (Distance): 3.5 km

Nodes Explored: 3

Theory Questions - (Find answer, justify them, if needed solve on paper and add images in your document. Don't need to write answers on paper. Directly add in documentation)

1. Which algorithm would you use for the following applications? Explain why. •
GPS Navigation
• File System Search
• Maze Solving
• Emergency Ambulance Routing
• Web Crawling

Application	Best Algorithm	Reason
GPS Navigation	A* Search	A* uses both actual cost and heuristic value to find the fastest route.
File System Search	DFS (Depth First Search)	DFS explores one directory completely before moving to another.
Maze Solving	BFS (Breadth First Search)	BFS guarantees the shortest path in an unweighted maze.
Emergency Ambulance Routing	Uniform Cost Search (UCS) or A*	Finds the least-cost or fastest route.
Web Crawling	BFS	Explores linked webpages level by level

2. Why do different algorithms behave differently even on the same graph?

Although the graph and goal remain the same, each algorithm uses a different data structure.

- **BFS** uses a Queue (FIFO).
- **DFS** uses a Stack (LIFO).
- **UCS** uses a Priority Queue based on path cost.
- **Greedy Best First Search** uses a Priority Queue based on heuristic value.
- **A*** uses a Priority Queue based on **f(n)=g(n)+h(n)**.

Therefore, each algorithm explores nodes in a different order and may produce different performance.

3. Difference Between Shortest Path, Least-Cost Path and Fastest Path

Term	Meaning	Example
Shortest Path	Minimum distance	3.5 km route from MVLU College to Sai Nagar
Least-Cost Path	Minimum total cost	UCS finds the minimum-cost route
Fastest Path	Minimum travel time	Google Maps considers traffic and road conditions

4. True or False

BFS always finds the least-cost path.

False

Reason:

BFS finds the path with the fewest edges, not the minimum cost.

DFS always finds the shortest path.

False

Reason:

DFS explores one branch completely before checking another.

UCS always finds the minimum-cost path.

True

Reason:

UCS always expands the node with the lowest accumulated cost.

Greedy Search is always optimal.

False

Reason:

Greedy Search only considers heuristic value and ignores actual path cost.

A Search uses both cost and heuristic values.*

True

Reason:

A* evaluates each node using

f(n)=g(n)+h(n)

where

- **g(n)** = actual cost
- **h(n)** = estimated cost

By combining both values, A* finds an efficient and optimal path.